Group 15 Experiment Report

Experiment 3 Desktop Object Classification and Sorting

Simulation Workflow and Physical Sorting Verification

Course	Robot Integration Group Project I
University	Ocean University of China
Class / group	Group 15
Report type	Group experiment report
Platform	Jetson, ROS 2, and mechArm 270
Task	Six source grids and two sorting areas

Revised September 22, 2026

Abstract

We built a desktop sorting system for the mechArm 270 using six pickup grids and two placement areas. An overhead camera identifies the object in each grid, and a ROS 2 state machine selects and executes the corresponding taught motion.

The Gazebo and MoveIt simulation uses HSV detection to sort four cubes and two hollow bridge-shaped objects. All six objects reached their target positions, including both bridges. The run took 391.53 s; the monitor recorded 37,831 joint samples with no joint-limit violations or unsafe contact pairs. Grasping uses a fixed attachment constraint after alignment and gripper-closure checks.

The physical system runs YOLO11 on a Jetson. Four cube classes go to the right placement area, while red cuboids and orange bridges go to the left. We use seven-frame histories with five matching group observations to stabilize grid decisions, and 17 taught poses for pickup and placement. The physical demonstration completed six transfers from five initial objects and one yellow-cube re-feed, with all six destination choices correct. Separate empty-cell and unknown-object tests checked the controller's no-motion branch.

The v4 detector's recorded mAP50–95 is 0.8755. A red/pink classifier assists the original detector, and a later red/orange classifier addresses confusion between the cuboid and bridge. The latter achieved 84/84 correct test crops and 36/36 detections in the saved video regression. These image tests are reported separately from robot transfer results.

Keywords: object sorting; YOLO11; ROS 2; mechArm; fixed-grid picking; Gazebo

Contents

Abstract **I**
1 Experiment Objectives and Requirements **1**
1.1 Purpose and connection with Experiment 2 . . . 1
1.2 Basic task and required interfaces . . . 1
1.3 Simulation and physical requirements . . . 1
1.4 Experimental setup . . . 2
2 System Design and Working Environment **2**
2.1 Hardware and communication . . . 2
2.2 Software modules and information flow . . . 3
2.3 Deployment, configuration, and module integration . . . 4
2.4 Six labels and two destinations . . . 4
3 Simulation **6**
3.1 Robot model and scene . . . 6
3.2 Scene, image detection, and grid mapping . . . 6
3.3 Automatic control and motion feedback . . . 7
3.4 Final two-shape acceptance . . . 7
3.5 Historical normal and exception runs . . . 7
3.6 Debugging and remaining work . . . 8
4 Visual Detection and Grid Mapping **9**
4.1 Six-class detection and training method . . . 9
4.2 A second check for red and pink objects . . . 10
4.3 Calibrating six image regions . . . 10
4.4 From a detection box to a pickup cell . . . 11
4.5 Stable decisions across several frames . . . 11
5 Taught Poses and Robot Motion **12**
5.1 Recording the physical pickup and placement poses . . . 12
5.2 Small joint steps and the transfer sequence . . . 13
5.3 TCP checks and clearance . . . 13
5.4 Class-specific gripping and delayed feedback . . . 14
5.5 Optional direction compensation . . . 14
6 Task Control and Exception Handling **14**
6.1 From a visual state to one transfer . . . 14
6.2 Checking after a transfer . . . 15
6.3 Exception states and recovery . . . 16
6.4 Unknown-object tests . . . 16
7 Experimental Procedure and Configuration **16**
7.1 Preparation and calibration . . . 16
7.2 Motion preparation and system startup . . . 17
7.3 Current physical Launch settings . . . 17
7.4 Records for analysis . . . 18
8 Results and Analysis **18**
8.1 Evaluation measures . . . 18
8.2 Simulation workflow results . . . 19
8.3 Offline model results . . . 19
8.4 Physical sorting demonstration . . . 19
8.5 Original task records . . . 20
8.6 Empty and unknown targets . . . 20
8.7 Red-cuboid and orange-bridge refinement . . . 20
8.8 Operating settings . . . 21
9 Main Problems and Solutions **21**
10 Conclusion **21**
A Physical Sorting Cycle **22**
B Visual Processing Logic **23**
C Submission File Index **24**
C.1 Reading the results . . . 24
C.2 Configuration and use . . . 25
References **26**

1 Experiment Objectives and Requirements

1.1 Purpose and connection with Experiment 2

The aim of Experiment 3 is to make the robot choose a useful action from camera information. In Experiment 2, the main work was recording robot states and replaying a fixed route. The operator already knew the picking and placing points. In this experiment, an object may appear in any of six prepared grids, and its group decides its destination. The system must connect visual detection, task decisions, and robot movement into one repeated process. [1, 3]

The task is to identify at least two object groups, select their source grids and place them in the corresponding areas. We use fixed pickup positions, so the camera only needs to identify a cell. The camera, robot base and objects must remain close to the calibrated arrangement.

1.2 Basic task and required interfaces

The handout requires four to six fixed source grids and at least two sorting areas. Our implementation uses six grids and two destinations. After the task starts, the operator must not manually choose the object class, source grid, or placement area. [1]

Table 1: Required interfaces and their roles

Interface	Required form	Purpose
Camera image	`sensor_msgs/Image`	Provide the scene used for detection.
Object detection	`vision_msgs/Detection2DArray`	Provide class, box, and confidence.
Grasping task	ROS 2 Action or an equivalent long task interface with feedback	Send a request and report progress or failure.
Task decisions	State machine or BehaviorTree	Select targets and manage normal and abnormal states.
System startup	One Launch file	Start the camera, perception, robot, and task control chain.

The task loop is to observe the table, find a valid object, decide its group and destination, execute picking and placing, and then observe again. Empty grids and unknown classes should be skipped. A failed grasp or unreachable target should lead to a stop or a safe return. Logs should preserve the recognition result, motion result, placement result, and error information.

1.3 Simulation and physical requirements

The required order is simulation first, followed by physical verification. Simulation should contain the arm, gripper, overhead camera, table grid, and at least two object groups. The simulated image must pass through the detection and task-control chain. The physical stage uses a fixed RGB camera, a Jetson computer, and the real mechArm. The physical layout must provide six target objects from at least two groups. [1]

Table 2: Acceptance targets stated in the Experiment 3 handout

Item	Simulation	Physical experiment
Sorting performance	At least 5 correctly sorted objects from at least 6 tested objects	At least 5 correctly sorted objects out of 6
Class coverage	At least 2 sorting groups	At least 2 sorting groups
Autonomy	No manual class, grid, or destination choice after startup	No manual class decision or action triggering during normal operation
Abnormal cases	Correctly handle at least 2 types	At least empty grids and unknown objects
Motion safety	No clear collision or joint-limit violation	No collision, limit violation, or dangerous action; safe stop on errors
Records and startup	Recognition, grasp, and state logs; one Launch	Per-object outcomes and errors; one Launch

The required submission includes the ROS 2 project, Launch and dependency files, trained weights, class definitions, grid configuration, task-control description, simulation and physical videos, test records, and this report. The task records link each selected target to its motion request and outcome.

1.4 Experimental setup

Our physical system uses six source cells and two destinations. The instructor accepted a test starting with five objects and re-feeding the yellow cube once, giving six transfers in total. The final simulation contains four cubes and two bridges. Its normal run and the earlier exception tests are described in Section 3. [18, 19, 16]

2 System Design and Working Environment

The hardware, labels, and interfaces in this section describe the physical system. The separate final simulation uses HSV detection, four red/blue cubes, two orange bridges, and MoveIt execution as detailed in Section 3; its topics and class mapping should not be substituted for the physical configuration.

2.1 Hardware and communication

The system has three main parts. The Windows computer starts the experiment and shows the camera preview. The Jetson runs the detector, the ROS 2 task controller, and the robot worker. The mechArm performs the joint movements and gripper actions. Keeping the decision and control programs on the Jetson avoids sending every small movement through the Windows preview connection. [2, 11]

The supplied launcher connects to the Jetson at `192.168.55.1`. The worker reaches the arm through the TCP endpoint `192.168.100.2:9000`. These addresses describe the laboratory arrangement, rather than a general requirement of the algorithm. The camera image size used for region calibration is 1280×720 pixels.

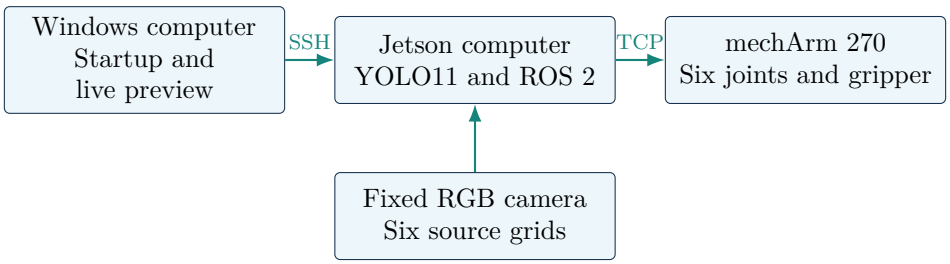


Figure 1: Hardware and communication structure. The drawing summarizes the deployment shown in the defense presentation.

The real scene contains colored objects, marked cells, the gripper, and background items such as cables and a power adapter. Cables and the adapter sometimes resemble targets and were included in the background training examples. The real photographs used in this report come from the Experiment 3 slides.

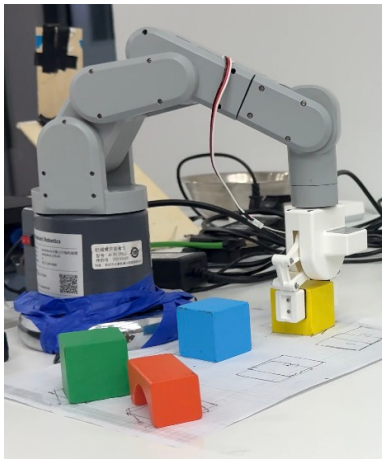


Figure 2: The mechArm and the tabletop objects in the supplied defense material, slide 7. The image shows the physical setup; it is not a measurement of position accuracy.

2.2 Software modules and information flow

The complete physical Launch file starts three ROS 2 nodes. The first node opens the camera, runs detection, publishes the raw image and detection array, and creates the preview and optional video. The second node maps detections to grids and runs the state machine. The third node receives one transfer request at a time and controls the arm. [11, 6]

Table 3: Main software modules in the final physical pipeline

File or module	Main responsibility
`detector_node.py`	Camera input, YOLO boxes and labels, red/pink refinement, ROS publication, preview and video.
`experiment3_task_controller.py`	Grid assignment, stable group states, target selection, request tracking, timeout and task logs.
`experiment3_robot_worker.py`	Validate requests, run the transfer process, and return progress and completion feedback.
`experiment3_transfer_test.py`	Build the selected taught route, move the arm, check position and gripper feedback, and recover on errors.
`experiment3_region_calibrator.py`	Create and check the six image polygons.
`experiment3_pose_recorder.py`	Record and save the named physical poses used by the six-grid task.

The image topic is `/camera/image_raw`, and the detection topic is `/yolo/detections`. A transfer request is sent on `/experiment3/transfer_requests`; worker responses return on `/experiment3/transfer_feedback`. The last two topics use JSON text inside ROS string messages. A request contains a request ID, grid ID, the detected six-class label after refinement, sorting group, destination, and detection confidence. The feedback links its status to the same request ID.

We use this request-and-feedback pair as the long-running task interface. It is implemented with topics rather than a ROS 2 Action server. The controller matches results by request ID and handles stale feedback and timeouts.

2.3 Deployment, configuration, and module integration

The supplied deployment process begins on Windows with configuration backup. The synchronization script downloads the active region polygons, gripper profiles, and 17-pose file from the original Jetson into the local package. This preserves the device settings before deployment. Changing the camera, robot base or gripper installation requires recalibration and pose checks. [11, 5, 7]

The deployment script transfers the two ROS 2 packages, `yolo_detection_ros2` and `mecharm_pick_place`, into the Jetson workspace. It also uploads the hardware helpers, test scripts, model weights, and configuration files. On the Jetson, it sources ROS 2 Humble and runs `colcon build` with `--symlink-install` for these two packages. The startup script later sources both the ROS environment and the workspace installation, so ROS can find the built nodes. [11]

The complete Launch file defines the active model, regions, poses, and gripper-profile paths in one place. The detector and controller receive the same region-file argument. This avoids displaying one set of grids while selecting actions from another. The active pose file and gripper profile belong to the same physical setup.

The three node names are `experiment3_yolo_detector`, `experiment3_task_controller`, and `experiment3_robot_worker`. Their topic names provide fixed connections between modules. Windows opens an SSH tunnel from local port 18091 to Jetson preview port 8091, allowing the browser to show the remote image. The launcher checks for an existing robot controller and asks for one `RUN AUTO` entry before setting `armed:=true`. The Launch default is disarmed. [11, 6]

Shutdown is also coordinated. Exit of the detector, controller, or worker triggers shutdown of the full Launch session. During cleanup, the worker attempts to stop an active runner, while the detector releases the camera and video resources. [11, 6, 4]

A clear integration sequence is to check the connection and active files, build the packages, inspect the camera and grids in preview mode, and then check the taught routes and gripper settings through the dedicated tools. The complete run follows these module checks. Timestamped video and task logs then link visual decisions to requests and worker feedback. [11, 8]

2.4 Six labels and two destinations

The detector retains six labels because the objects have different colors and shapes. The task controller reduces them to two movement groups. The distinction matters when a label is wrong. A blue-to-green cube error keeps the same destination. A pink-cube-to-red-cuboid error changes the destination. [4, 9]

Table 4: Class mapping and gripper targets

Detector label	Group	Destination	Closing target
`blue_cube`	cube	Right	69
`green_cube`	cube	Right	69
`pink_cube`	cube	Right	69
`yellow_cube`	cube	Right	69
`red_cuboid`	other	Left	40
`orange_bridge`	other	Left	40

Both groups use an opening target of 100 and a gripper speed setting of 20. These values are device command values in the supplied profile. They are control values, not calibrated measurements of jaw angle or force. The profile is loaded from a configuration file so that a change in the gripper setup does not require rewriting the task logic.

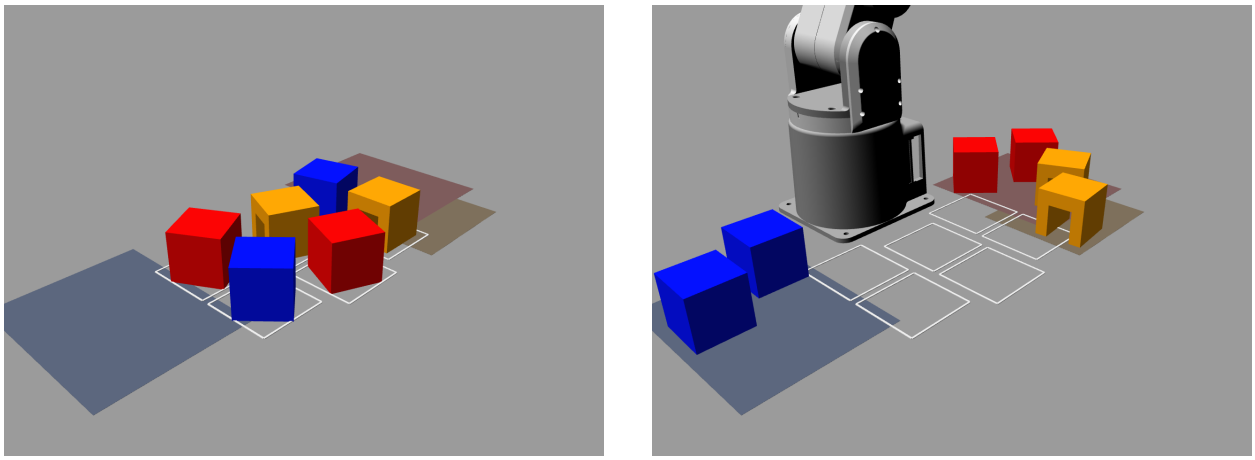
3 Simulation

3.1 Robot model and scene

The earlier Gazebo and MoveIt fixed-point pick-and-place project provided the development base. It established the robot model, collision scene, grasp approach, carrying motion, release, and return. Keyframe teaching and replay supported motion development before the six-grid task was integrated. [13]

The original `experiment3_sim_complete_20260914` used six red/blue cubes. The final extension, `experiment3_sim_bridge_20260921`, replaces the red cube in G2 and blue cube in G4 with two orange bridges. The six grid coordinates, other four objects, red/blue detection rules, original red/blue slots, and 65 mm carrying-centre height remain unchanged. The final scene has two red cubes, two blue cubes, and two bridges. [15, 18]

Each bridge is one 50 g rigid body with a crossbeam and two legs, three physical collision boxes, and a through-opening. Its envelope is $40 \times 40 \times 40$ mm; the opening is 20 mm wide and 28 mm high. Centre of mass and inertia are calculated from the component volumes. MoveIt conservatively uses the outer box for collision checking.



(a) Four cubes and two bridges in six source grids. **(b)** Final placement after six successful transfers.

Figure 3: Actual Gazebo side-camera images from the final bridge run. This observer camera does not participate in task decisions.

3.2 Scene, image detection, and grid mapping

The scene contains the mechArm, gripper, table, six fixed source cells, red/blue destination slots, added bridge slots on the left, and an overhead RGB sensor. The camera publishes a 960×720 image on `/task3/camera/overhead/image_raw`. The configured image rate is 8 Hz, not a measured processing rate. [18]

The final simulation uses HSV segmentation and contour filtering for `red_cube`, `blue_cube`, and `orange_bridge`. It publishes class labels, boxes, and scores as `vision_msgs/Detection2DArray` on `/task3/detections`. Detection uses camera pixels, not scene ground-truth labels. Box centres are matched to projected grid regions and colors select destinations. This is color-associated classification, not color-independent shape recognition. [18]

The simulation requires four consecutive matching grid observations and uses a configured minimum score of 0.65 before selecting a target. These settings belong to the HSV simulation pipeline. The physical system instead uses YOLO11 six-class detection, red/pink refinement, calibrated image polygons, and five-out-of-seven group voting. The two detectors and their scores are not interchangeable.

3.3 Automatic control and motion feedback

The simulation state machine contains `WAIT_READY`, `OBSERVE`, `SELECT`, `EXECUTING`, `VERIFY`, `RECOVER`, `DONE`, and `STOPPED`. The controller selects an eligible grid, submits one grasp job, waits for the result, allows the scene to settle, and then observes the remaining cells. The configured normal run processes each of the six grids once. It does not use the physical controller's refill policy. [15]

The grasp bridge exchanges request, status, result, and cancellation messages with job IDs. This is an equivalent feedback-based long-running task interface, rather than a standard ROS 2 Action server. Motion feedback and a 180 s controller job timeout prevent the controller from issuing another transfer while an earlier one is outstanding.

MoveIt plans the approach, grasp, lift, transfer, release, and return motions with collision objects. A simulation grasp check verifies alignment and gripper closure before attaching the object with a fixed constraint. This attachment supports carrying without snapping the object to a new pose; it is not a measurement of real friction or gripping force. Joint-state sampling and monitored physical contacts provide additional run checks. [15]

The complete system starts from one `full_sorting.launch.py`, including the scene, robot controllers, planning system, detector, grasp bridge, task controller, and independent acceptance monitor. The normal run does not require manual class, source-cell, or destination selection after startup.

3.4 Final two-shape acceptance

The final result is saved in `results/bridge_20260921_222016/acceptance.json`. Independent evaluation of actual Gazebo positions reports `PASS` and `DONE`: all six objects, including both bridges, are correctly placed. The evaluator checks the final position of each object. [19]

Table 5: Object-level results of the final bridge simulation

Grid	Object	Target XY (m)	Independent result
G1	Blue cube	$(0.170, 0.070)$	Correct
G2	Orange bridge	$(-0.140, 0.125)$	Correct
G3	Red cube	$(-0.170, 0.070)$	Correct
G4	Orange bridge	$(-0.094, 0.164)$	Correct
G5	Red cube	$(-0.112, 0.045)$	Correct
G6	Blue cube	$(0.112, 0.045)$	Correct

The final run records 37,831 joint samples, zero joint-limit violations, zero monitored unsafe contact pairs, zero failed jobs, and a wall-clock duration of 391.53 s. The second bridge slot centre is just outside the source-grid edge, next to G4 after it has been emptied. Existing red/blue routing is preserved and remains different from the physical cube/other mapping.

Saved evidence includes configuration, world geometry, detections, job results, motion and safety logs, and two videos. The overhead video contains 1,354 frames encoded at 8 fps; the side video contains 651 frames at 5 fps. The videos use their configured encoding rates; the task duration comes from the run log.

3.5 Historical normal and exception runs

The normal record is `results/normal_front_arc_04/acceptance.json`. Independent evaluation of final object positions reports all six objects in their color-specific destinations: three red cubes and three blue cubes. The run finishes in `DONE`, with 37,412 joint samples, no recorded joint-

limit violations, and zero contact pairs classified as unsafe by the monitor. Its recorded wall-clock duration is 466.26 s. [16]

Table 6: Recorded simulation results for the configured same-shape scene

Item	Normal run	Exception run
Run directory	`normal_front_arc_04`	`exceptions_front_arc_03`
Task state / evaluator status	`DONE` / `PASS`	`DONE` / `PASS`
Correct placements	6 of 6 color-labeled cubes	2 normal targets sorted; designated exception cells handled
Joint-state samples	37,412	18,069
Recorded joint-limit violations	0	0
Monitored unsafe contact pairs	0	0
Wall-clock duration	466.26 s	238.73 s

The exception record is `results/exceptions_front_arc_03/acceptance.json`. Grid 2 contains an unknown-color object, grid 3 is empty, grid 5 has an injected unreachable-target condition, and grid 6 has an injected grasp failure. The controller skips the unknown and empty cells and handles the two motion failures through recovery. The remaining two ordinary targets are sorted. This run tests the designated exception branches; its two placements are not a normal-task sorting accuracy of two out of six. [16]

Both run directories preserve detections, task states, motion events, grasp-interface records, safety logs, initial/final images, and video. The independent evaluator checks actual final object positions rather than relying only on the controller's success count.

3.6 Debugging and remaining work

The first bridge tests exposed an unreachable placement point and a Gazebo crash during release. We moved constraint operations to the physics-update thread and checked the distant slot with collision-free IK queries. Moving that slot inward allowed all six transfers to finish without lowering the carrying height. [18]

After the accepted run, `move_group` logged a segmentation fault during SIGINT shutdown, while Gazebo exited normally. This shutdown issue remains to be resolved. The earlier exception tests cover the original scene; the bridge version has completed the normal run but has not repeated those exception tests.

4 Visual Detection and Grid Mapping

The detector, model metrics, and grid processing in this section apply to the physical experiment. The recorded simulation uses HSV color segmentation instead, as described in Section 3.

4.1 Six-class detection and training method

The vision system answers two questions before the robot moves: what kind of object is present, and which pickup cell contains it? We used YOLO11 to predict a class, a bounding box, and a confidence score for each object. The six labels were `blue_cube`, `green_cube`, `pink_cube`, `yellow_cube`, `red_cuboid`, and `orange_bridge`. The detector kept these labels for inspection, while the task controller reduced them to two sorting groups. The four cube classes went to the right area. The red cuboid and orange bridge went to the left area. In this project, "v4" means the fourth project model version, not the YOLOv4 network. [2, 4]

The defense slides describe training with a 640×640 input, the AdamW optimizer, light data augmentation, and 35 epochs. Each training object had a class label and a box. The positive samples covered different colors, shapes, and views. We also included difficult background samples, such as an empty desk, cables, and the robot body. These negative samples helped the detector learn when it should produce no object box. This was useful because a background error could otherwise create a false pickup request. [2]

The training, validation, and test sets had different roles. Training changed the model weights; validation helped select the saved weights; the test set checked detection quality after training. The selected model was then used for CUDA inference on the Jetson. Model evaluation results are given in Section 8. [2]

Data quality was as important as the training settings. Boxes need to cover the object consistently, and labels need special care for red/pink views, touching objects, and partial occlusion. Background images should resemble the actual work area. Video frames close in time can be very similar, so a test on separately recorded scenes gives stronger evidence of general use than a random split of nearby frames. The supplied earlier data-building script uses time blocks for splitting; the final v4 dataset size is not stated here because a matching final dataset summary was not supplied. [4]

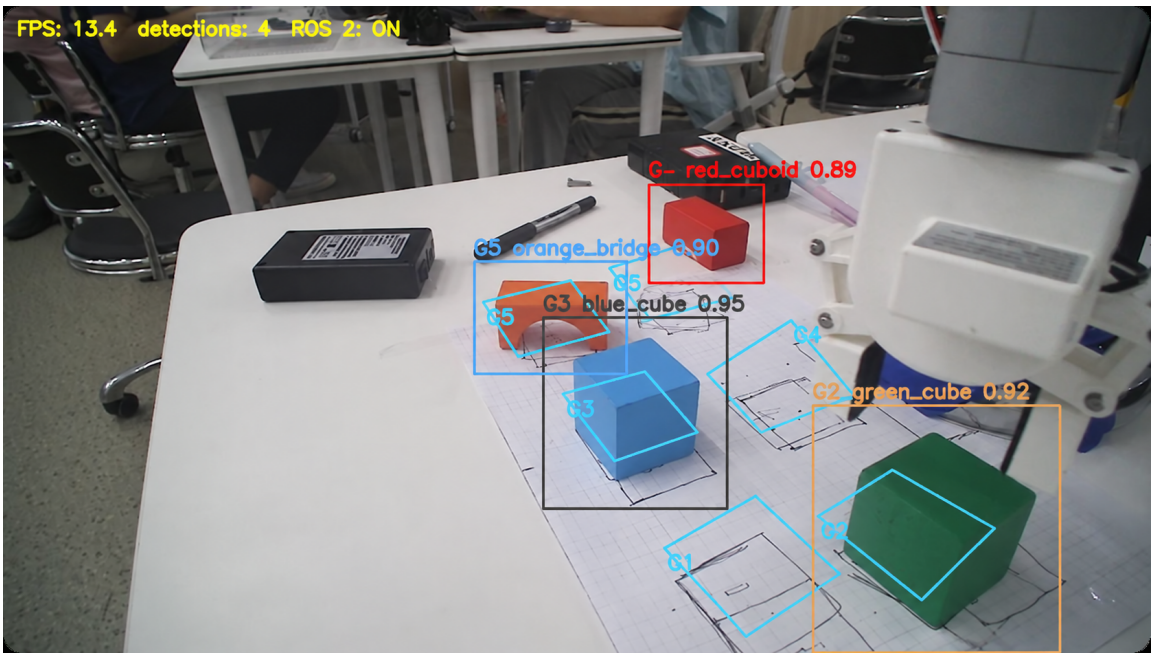


Figure 4: Detection and grid overlays in the experimental scene, reproduced from defense slide 4. Boxes show class names and confidence scores; blue polygons mark pickup regions. The displayed 13.4 FPS is one snapshot value, not a measured average over a complete run. [2]

4.2 A second check for red and pink objects

The red cuboid and pink cube were a difficult pair. Their colors could look similar under the camera, and a cuboid viewed from one end could appear nearly square. An error between these classes also changed the destination, so it deserved special attention. We therefore used the separate `red_pink_shape_classifier_v2.pt` model to check the object crop after the main detection. [4]

For a pink or red detection, the program added padding equal to 8% of the box width and height on each side. It kept the crop inside the image and passed it to the second model with an input size of 224. The correction rule was one way: an original pink-cube label changed to red cuboid only if the second model predicted red with a score of at least 0.55. An original red label was kept. The box position and main detector confidence were also kept. [4]

The second classifier only processes candidate crops from the main detector, so it cannot recover an object that YOLO misses.

4.3 Calibrating six image regions

The table had six fixed pickup cells, named G1 to G6. Each cell was linked to taught robot poses. The camera therefore needed to select the correct cell, rather than calculate a new three-dimensional grasp pose for every object. The calibration program represented each cell by a four-corner polygon in the 1280×720 camera image. The operator clicked its corners in the browser view and then tested points inside and outside the regions. [5]

Before saving, the program checked that all six IDs were present, the corners were distinct and inside the image, and each polygon formed a valid convex quadrilateral. It rejected regions with an area below 1,000 square pixels and overlaps larger than two square pixels. These checks caught incomplete or poorly drawn regions before they reached the controller. The accepted polygons were stored in `regions.yaml`. [5]

The camera and table layout needed to stay fixed after this step. A change in camera position could move a visible cell even though its stored polygon stayed unchanged. Likewise, moving the

robot base could break the link between a cell and its taught pose. The empty example configuration was only a template; the real experiment required the measured regions for its own setup.

4.4 From a detection box to a pickup cell

Let the top-left and bottom-right corners of a detected box be (x_1, y_1) and (x_2, y_2). Its image center is

$$\mathbf{c} = (c_x, c_y) = \left(\frac{x_1 + x_2}{2}, \frac{y_1 + y_2}{2} \right). \tag{1}$$

For region polygon P_i, define the set of matching cells as

$$\mathcal{G}(\mathbf{c}) = \{ i \in \{1, \ldots, 6\} : \mathbf{c} \in P_i \}. \tag{2}$$

The program assigns a grid ID only when $|\mathcal{G}(\mathbf{c})| = 1$. A center outside all regions has no pickup cell. A center assigned to more than one region is also left unresolved. The controller uses a point-in-polygon calculation for this decision. [4, 6]

Vision selects the cell, and the arm uses its stored approach and grasp poses. Objects need to stay near the taught pickup positions because this method does not estimate depth or adjust the grasp within a cell.

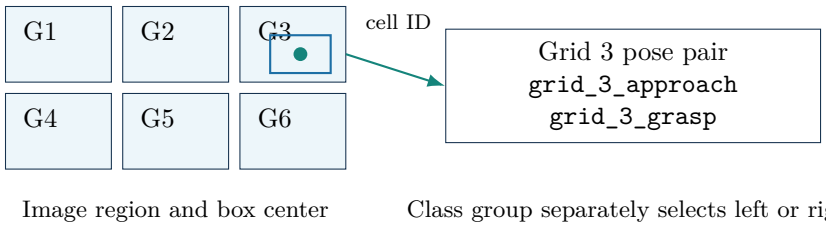


Figure 5: Discrete image-to-pose mapping. Grid 3 is an illustrative example; the drawing is a schematic, not the measured camera calibration.

4.5 Stable decisions across several frames

A box in one frame was not enough to trigger a pick. For each cell, the controller first converted the current detections into a sorting observation. No box gave `EMPTY`; an unsupported class gave `UNKNOWN`; and boxes from different sorting groups gave `MULTIPLE`. These unresolved cases did not become pickup targets. [6]

The formal ROS controller treated several boxes from the same group as one group observation and used the highest-confidence box as its representative. This addressed cases where one physical cube produced overlapping boxes with different color labels. Since all cube labels shared a destination, those duplicate labels did not need different actions. The method still assumed one physical object per pickup cell: it could not distinguish duplicate boxes from two real objects of the same group. [6]

The controller then stored the last seven observations, $o_{t-6}, \ldots, o_t$. For each possible group or state s, it counted

$$N_t(s) = \sum_{k=t-6}^{t} \mathbf{1}[o_k = s]. \tag{3}$$

After the window was full, a state was stable if it received at least five votes. Stable cube and other observations became `READY_CUBE` and `READY_OTHER`. Stable `EMPTY`, `UNKNOWN`, and `MULTIPLE` states were kept. If no state received five votes, the cell remained a candidate. Before issuing a request, the controller also required the current group to agree with the stable ready state. Voting was thus based on the sorting group, not an exact color label. [6]

We set the detector confidence threshold to 0.25. Unique center-in-polygon assignment determines the source grid, and the controller requires five matching sorting-group observations in

a seven-observation window. These checks suppress brief detection changes before a new target becomes ready. [4, 6]

The confidence threshold, the NMS overlap threshold, and the temporal vote solve different problems. Confidence controls which candidate boxes are kept. NMS, configured with an IoU threshold of 0.45, reduces overlapping detections. The vote compares observations across frames. A lower confidence threshold can retain weak targets but also retain more false boxes. Persistent false detections can still pass these checks.

5 Taught Poses and Robot Motion

The taught poses, robot feedback, and gripper settings below belong to physical execution. Simulation uses the separate MoveIt and grasp-constraint workflow in Section 3.

5.1 Recording the physical pickup and placement poses

The fixed cells allowed us to use a small set of taught robot poses. After vision selected a cell and a sorting group, the worker loaded its pickup poses and the correct destination poses. This reduced the need to estimate a new three-dimensional grasp point from each image. It also made the limits clear: the camera, robot base, grid sheet, and gripper had to stay in their calibrated positions, and each object had to be placed near its cell center. [7, 8]

The physical recorder guided the operator through 17 named poses. It stored six joint angles and the tool center point (TCP) position and orientation for each pose. The source approach poses were taught above the objects; the grasp poses placed the fingers at the required pickup height. Both destinations had an approach pose and a lower release pose. A shared safe pose linked the source and destination routes. [7]

Table 7: The 17 named poses used by the physical task

Location	Pose names	Count
Shared	`safe`	1
Grid 1	`grid_1_approach`, `grid_1_grasp`	2
Grid 2	`grid_2_approach`, `grid_2_grasp`	2
Grid 3	`grid_3_approach`, `grid_3_grasp`	2
Grid 4	`grid_4_approach`, `grid_4_grasp`	2
Grid 5	`grid_5_approach`, `grid_5_grasp`	2
Grid 6	`grid_6_approach`, `grid_6_grasp`	2
Left area	`left_approach`, `left_release`	2
Right area	`right_approach`, `right_release`	2
Total	One shared pose, twelve source poses, four destination poses	17

The formal transfer code required a complete pose file and an active correction record covering all 17 points. It checked the angle limits and rejected invalid values before motion. The inherited simulation tools used RViz and the `q`/`qa`/`qb` keyframe method. They were useful for learning and checking motion ideas, while the physical Experiment 3 route used the named-pose recorder above. The two formats should not be treated as the same dataset. [7, 8, 2]

5.2 Small joint steps and the transfer sequence

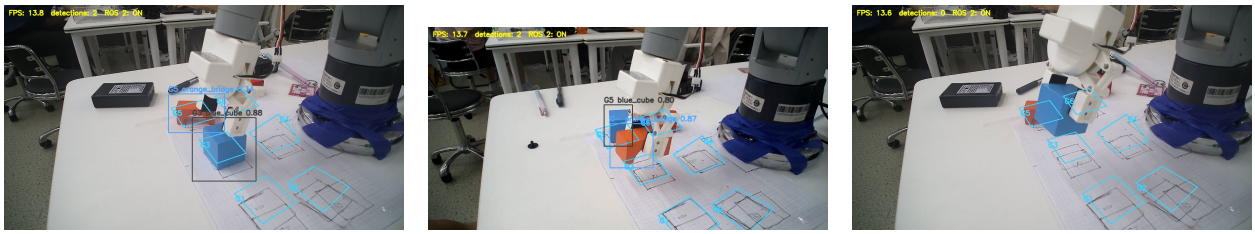
The robot moved between taught joint targets using linear interpolation. Let $\mathbf{q}_0$ and $\mathbf{q}_1$ be two six-joint vectors, and let δ be the largest allowed command step. The program used

$$N = \max\left(1, \left\lceil \frac{\max_j |q_{1,j} - q_{0,j}|}{\delta} \right\rceil\right), \qquad \mathbf{q}_k = \mathbf{q}_0 + \frac{k}{N}(\mathbf{q}_1 - \mathbf{q}_0), \quad k = 1, \ldots, N. \tag{4}$$

Thus, no joint command changed by more than δ within one step. The final launch set $\delta = 2°$, a command interval of 0.20 s, and an SDK arm-speed value of 8. These are software settings; 8 is not a measured speed in degrees per second. [8, 11]

The commands were sent in small steps, and feedback was checked at key poses. Joint interpolation did not guarantee a straight TCP path or impose a verified acceleration limit. We therefore relied on the taught layout, safe transition poses, and measured endpoint checks. [8]

A transfer began at the safe pose with the gripper open. The arm moved above the selected cell, checked the open gripper, descended, and closed to the class-specific setting. After lifting, it checked the grip and returned through the safe pose. It then approached the selected destination, descended, released the object, lifted clear, and returned to safe. Figure 6 shows three physical stages from the defense presentation. [8, 2]



(a) Approach the source. **(b)** Descend and grasp. **(c)** Lift the object.

Figure 6: Physical motion stages shown in the supplied defense slides. These images illustrate the sequence; they do not provide a measured success rate. [2]

5.3 TCP checks and clearance

Sending a target did not mean that the arm had reached it. Before closing, the program waited for the TCP to settle near the grasp point. It sent the grasp route once, then read feedback without repeatedly sending the same joint target. Two consecutive acceptable samples were required. This reduced the chance of closing while the old arm was still moving. [8]

Table 8: Selected motion acceptance settings, not measured positioning accuracy

Check	Configured condition
Ordinary grasp point	TCP horizontal error ≤ 4 mm; vertical error ≤ 3 mm.
Manually corrected grasp point	TCP horizontal error ≤ 5 mm; vertical error ≤ 8 mm.
Grasp stability	Two consecutive valid samples; up to nine checks at 0.75 s intervals.
Grasp joint guard	Other joint errors $\leq 3°$; joint 5 error $\leq 8°$.
Generated destination approach	Minimum clearance 50 mm; target 65 mm, also considering the highest recorded TCP position.
Post-release retreat	Measured TCP lift at least 75 mm, with a 0.5 mm feedback allowance.

The final code also adjusted difficult cells. It raised the requested grasp positions for grids 3 and 6 by 7 mm and 10 mm along their taught paths, and extended the grid 3 approach by 20 mm. These were generated targets, checked again through feedback. Since joint and Cartesian motion are not identical, the requested height change alone did not prove the physical result. [8]

At the destination, the code could extend the taught release-to-approach direction to create a higher transition. After release, it checked the actual TCP height gain before returning across the workspace. These checks addressed low approach poses and the risk of touching the released object during departure. They still required a clear route and did not replace full obstacle detection. [8]

5.4 Class-specific gripping and delayed feedback

One closing setting was not suitable for both object groups. The saved profile used OPEN=100, cube CLOSE=69, other CLOSE=40, and speed=20. These values use the gripper's control scale from 0 to 100; they are not angles or measured grip forces. The loader checked that the two close values were below the open value and that the profile was complete. [9]

The old gripper could respond late or stop briefly before reaching its final position. The program therefore checked visible feedback movement and several stable readings. For cube closing, the normal accepted feedback range was 64–72. A value of 73 was accepted only through a longer stable-contact check, requiring eight stable samples and enough closing movement. A brief pause at a more open position did not pass this check. [8, 9]

After lifting and before transfer, the program read the gripper position again without sending another close command. If an actuation failed, it could clear old commands and rebuild the control connection, with at most three sessions. Continued failure stopped the sequence. Position feedback was useful, but it could not measure grip force or prove that every object remained secure. [8, 9]

5.5 Optional direction compensation

An additional tool supported direction-based joint correction. For a joint approaching a pose from increasing or decreasing angles, it selected a matching correction and formed $q_{\mathrm{cmd},j} = q_{\mathrm{taught},j} + c_{j,d}$. This addressed the possibility that joint play and load caused different errors for different approach directions. When a joint target did not change, the previous direction was retained. [10]

The tool required validated settings, limited each correction to 3°, and supported a small preload movement. However, the supplied formal ROS worker did not enable this optional configuration by default. The main physical method was the corrected 17-pose route with feedback checks; the existence of the compensation tool does not establish a measured improvement in accuracy. [8, 10]

6 Task Control and Exception Handling

This section describes the physical controller, including its five-out-of-seven decisions and refill handling. The simulation controller uses four consecutive matching observations and a single pass over the six source grids; its state machine is described in Section 3.

6.1 From a visual state to one transfer

Each grid has a current visual state. A stable cube group becomes `READY_CUBE`; a stable non-cube group becomes `READY_OTHER`. Other states include `EMPTY`, `UNKNOWN`, `MULTIPLE`, and `CANDIDATE`. The controller chooses a ready target by ascending grid number. The current representative group must agree with the stable ready state. This prevents an old majority alone from creating a request when the current observation points to a different group. [6]

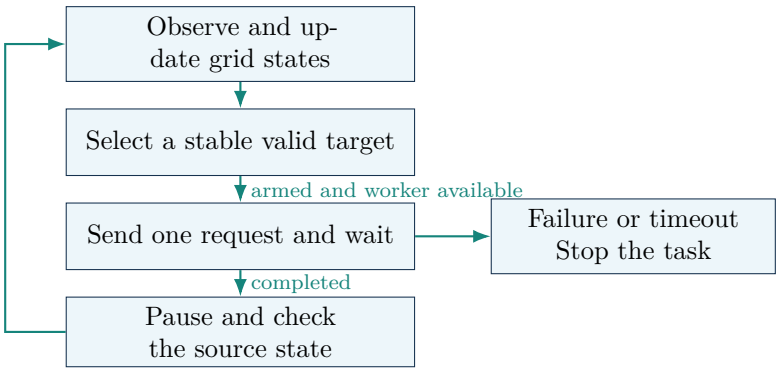


Figure 7: Simplified task cycle. Non-target states are skipped; final completion also requires a stable empty scene.

The controller permits only one outstanding request. The worker also has a busy flag and checks that the grid lies in 1–6 and that the group matches the destination. It returns `STARTED`, `COMPLETED`, or `FAILED` with the request ID. Feedback for a different request is ignored by the controller. The worker sends `COMPLETED` only after the transfer routine returns without an error; that status describes software execution and its configured checks.

6.2 Checking after a transfer

After a completed transfer, the controller waits for the five-second interval and checks the source grid again. An `EMPTY` state accepts the transfer. A stable ready state also accepts the previous transfer and queues the current object again. This second rule supports immediate replenishment of a source cell. [6]

A refilled cell and an unsuccessful pickup can produce the same ready state. We therefore checked placement in the video as well as the controller result.

The whole task ends automatically only after at least one transfer has been accepted, all six regions are stably empty, and that empty condition lasts for the configured five seconds. An initially empty table does not immediately finish the session. Unknown or mixed-group states also do not satisfy the all-empty completion condition, even though they do not trigger a pick.

6.3 Exception states and recovery

Table 9: Abnormal inputs and the implemented response

Condition	Controller or worker response
Empty cell	Record a skip and send no transfer request.
Unsupported class	Mark `UNKNOWN`, record a skip, and send no transfer request.
Conflicting groups in one cell	Mark `MULTIPLE`; do not select the cell.
Unstable observations	Keep `CANDIDATE` until a stable state is available.
Outside or ambiguous region	Assign no grid, so the detection cannot choose a route.
Disarmed mode or busy worker	Do not start a new movement; the worker rejects an invalid request.
Invalid pose, failed feedback check, or motion error	Raise a transfer error, attempt the programmed stop or recovery, and report failure.
Transfer exceeds 180 s	Record the timeout and stop the Launch through controller shutdown.

The complete Launch registers the exit of each main process. If the detector, controller, or worker exits, it requests shutdown of the remaining system. The worker attempts to stop an active runner during shutdown. A control lock also prevents two cooperating programs from commanding the same arm at once. Emergency-stop response time was not measured in these tests. [11, 8]

6.4 Unknown-object tests

The project contains two related tests. The interface test publishes an unsupported label, `unrecognized_object`, at a selected cell center. It checks whether the controller enters `UNKNOWN`, records `SKIP_NO_MOTION`, and creates no transfer request. This isolates the task-control rule from detector quality. [12]

The physical unknown test first records an empty-cell background and then a scene with the test object. It uses a brightness-adjusted image difference to check occupancy. Its default criteria require a changed area of at least 3.5% of the cell and a largest connected area of at least 2%. It also requires no known-class detection in that cell. If these conditions hold, the test publishes the same unsupported label. Both test paths run the controller in disarmed mode and check for zero movement requests.

The occupancy test runs separately from normal six-class detection. During normal sorting, an unfamiliar object can still be missed or assigned a known label.

7 Experimental Procedure and Configuration

7.1 Preparation and calibration

The procedure connects the separately tested visual and movement parts. It begins with a fixed work area: the camera and robot base should not move after calibration, each source cell should contain at most one physical target, and the transfer and release areas should remain clear. A changed camera view changes the pixel polygons, while a changed robot or gripper setup changes the taught poses. These are separate reasons for recalibration. [5, 7]

The deployment scripts distinguish the current device data from the software package. They provide a synchronization step for preserving the original Jetson's regions and poses before de-

ployment. This matters because an empty example file is not a usable calibration. The supplied `regions.example.yaml` contains no actual region coordinates. A correct setup must recover the working `regions.yaml` and pose file from the device or collect them again.

For the visual module, the preparation sequence is:

1. Check that the six class names in the model and the controller have the same spelling and meaning.
2. Fix the camera at the required view and use the 1280×720 image for region calibration.
3. Mark four corners for each of the six source cells. Check the region IDs, polygon shape, area, and separation.
4. Put known objects in the cells and review the preview. Check both the class result and the grid result; one can be correct while the other is wrong.
5. Observe the empty scene and the backgrounds likely to appear during motion. Review false boxes on the table, cables, gripper, and nearby objects.
6. Check the group states across several frames before enabling robot execution.

These steps describe the operating procedure supported by the tools. They do not add new test outcomes to the recorded defense results.

7.2 Motion preparation and system startup

The motion part needs the 17 named poses and the matching gripper profile. The route should first pass the available data checks and read-only precheck. Low-speed motion checks then establish whether the cell and destination poses match the physical arrangement. A successful visual preview alone does not validate reachability or clearance. [8, 7]

The complete physical launcher is `experiment3_real.launch.py`. Its default is `armed:=false`, which supports inspection before movement. The Windows `start_experiment3_ros2_auto.cmd` entry checks for an existing controller and asks for the initial `RUN AUTO` input. It then starts the complete ROS 2 Launch in armed mode. This initial enable step is distinct from manually selecting each object's class or route. [11]

During a normal cycle, the detector and controller choose the object, cell, and destination. The operator does not supply those decisions. The current launcher does, however, ask the operator to clear each released object during the five-second interval. The system supports automatic selection and transfer within this operating arrangement; it does not provide automatic stacking or management of a full destination area.

7.3 Current physical Launch settings

Table 10 records the supplied complete Launch configuration. The settings describe software inputs. They are not measured rates, motion accuracy, or a record of every value used during the earlier presentation run. [11]

Table 10: Settings in the supplied complete physical Launch

Setting	Value	Meaning
Camera image size	1280×720	Must match the region calibration.
Requested capture rate	30 frames/s	Camera request, not measured inference speed.
Detector input size	640	Input scale used by the detector.
Detector confidence	0.25	Candidate detection threshold.
NMS IoU threshold	0.45	Box-overlap threshold used in duplicate suppression.
Red/pink correction threshold	0.55	Required specialist score for the implemented pink-to-red change.
History and stable votes	7 and 5	Group/state voting settings.
Inter-object interval	5 s	Delay after a completed movement.
Empty-scene completion wait	5 s	Required stable idle time after work has occurred.
Transfer timeout	180 s	Maximum controller wait for one request.
Arm speed setting	8	Device motion setting used by the worker.
Maximum joint command step	2°	Bound used by joint interpolation.
Command period	0.20 s	Delay between streamed targets.

7.4 Records for analysis

The system can save an annotated video, JSONL task events, and a motion log for each transfer. The most useful record joins the request ID with the grid, predicted class, selected group, destination, worker result, and visible placement outcome. This makes it possible to trace an incorrect placement back to recognition, mapping, or movement. [11, 6]

For example, a correct label with a wrong grid suggests a spatial mapping problem. A correct grid with the wrong left/right choice suggests a class-to-group error. A correct request with an object still on the table suggests a physical execution problem. The present report uses this separation when interpreting results. New cycle times, per-object error values, and average frame rates are not estimated from isolated screenshots.

8 Results and Analysis

8.1 Evaluation measures

We evaluated image recognition, task execution and final placement separately. The detection metrics below describe model predictions; physical sorting results come from the videos and task records.

For detection, precision and recall can be written as

$$P = \frac{TP}{TP + FP}, \qquad R = \frac{TP}{TP + FN}, \tag{5}$$

where TP is a correct detection, FP is a false detection, and FN is a missed target under the evaluation matching rule. Precision helps describe false-target pressure; recall helps describe how many targets are found. Both are relevant to sorting. A false target may cause an unnecessary pick, while a missed object may remain on the table.

The overlap between a predicted box B and a reference box G is

$$\mathrm{IoU}(B, G) = \frac{|B \cap G|}{|B \cup G|}. \tag{6}$$

mAP50 summarizes average precision with an IoU matching threshold of 0.50. mAP50–95 averages results over thresholds from 0.50 to 0.95 in steps of 0.05. The detector's mAP and the robot's transfer success rate measure different stages of the system.

8.2 Simulation workflow results

The final HSV simulation sorted four cubes and two bridges correctly: 6/6 total and 2/2 bridges. Independent acceptance recorded 37,831 joint samples, no joint-limit violations, no monitored unsafe contact pairs, and 391.53 s. Section 3 provides all six object-level results and saved evidence. [19]

The two geometries completed the same pickup and placement sequence. The HSV detector and fixed-constraint grasp model are described in the simulation chapter.

8.3 Offline model results

The model results available from training and the defense presentation are listed below. The v3 baseline includes four metrics; the v4 result gives mAP50–95. [2]

Table 11: Detector results reported in the defense presentation

Model and source	Precision	Recall	mAP50	mAP50–95
v3 baseline, slide 9	0.9505	0.9342	0.9360	0.8699
Final v4, slide 4	Not stated	Not stated	Not stated	0.8755

The recorded mAP50–95 increased from 0.8699 to 0.8755, a difference of 0.56 percentage points. The v4 training added field images and difficult backgrounds. The available summary results do not isolate the contribution of individual training changes.

The red/pink classifier achieved 94.12% Top-1 accuracy on 68 test crops: 26 pink and 42 red. In the running pipeline, it changes a pink prediction to red only when its score reaches the configured threshold. The crop test does not include missed detections or grid-assignment errors. [14]

8.4 Physical sorting demonstration

With the instructor's approval, we started the physical test with five objects and re-fed the yellow cube once, producing six transfers. The acceptance video shows the red cuboid, yellow cube, re-fed yellow cube, blue cube, orange bridge, and green cube delivered to their corresponding sorting areas. Four cube transfers go right and two non-cube transfers go left. All six transfers went to the intended sorting area; the group recorded no failed transfer. [20]

$$S_{\text{cycles}} = \frac{6 \text{ matching destination choices}}{6 \text{ transfer cycles}} = 100\%. \tag{7}$$

Table 12: Observed acceptance-video cycles

Cycle	Visible object	Area	Grasp/transport	Delivery interval
P01	Red cuboid	Left	Observed	00:22–00:25
P02	Yellow cube	Right	Observed	01:16–01:19
P03	Re-fed yellow cube	Right	Observed	02:04–02:06
P04	Blue cube	Right	Observed	02:48–02:49
P05	Orange bridge	Left	Observed	03:22–03:24
P06	Green cube	Right	Observed	03:59–04:01

The intervals refer to `2eb7001d38fd861630d52918161ba8cf.mp4` and use approximate playback timestamps. The operator re-feeds the yellow cube and clears delivered objects. Cycle IDs describe transfer order rather than six distinct physical objects.

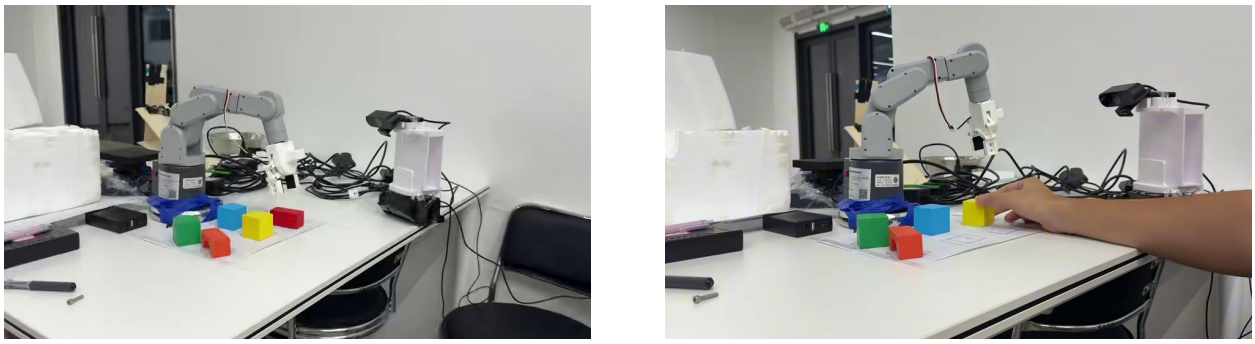


Figure 8: Physical acceptance: initial scene and yellow-cube re-feed at approximately 01:24.

8.5 Original task records

The task JSONL for 2026-09-08 03:25:15 records an armed controller, six transfer requests, six completed worker results, six accepted transfers, and final `completed_count=6`. Its chronological source grids are G1, G3, G2, G4, G5, and G6. Request IDs connect source selection, destination, worker feedback, and source-clear verification. [21]

Table 13: Six-cycle task log

Request	Grid	Recorded detector label	Area	Worker result
1	G1	`orange_bridge`	Left	COMPLETED
2	G3	`yellow_cube`	Right	COMPLETED
3	G2	`yellow_cube`	Right	COMPLETED
4	G4	`blue_cube`	Right	COMPLETED
5	G5	`orange_bridge`	Left	COMPLETED
6	G6	`green_cube`	Right	COMPLETED

The table lists detector outputs. The video table above instead describes visible object shapes and playback times; the source files are identified with each record.

The v4 task of 2026-09-07 19:26:01 is a separate five-transfer run, covering G1, G2, G3, G5, and G6. G4 remained without a stable target. Its final count is five. The matching visual-recognition AVI shows the detector overlays and five-object scene; these records are not added to the six-cycle total.

8.6 Empty and unknown targets

Recovered physical-unknown tests at 15:38:52 and 19:50:38 on September 7 record G1 as `UNKNOWN` and then `SKIP_NO_MOTION`, with no transfer request. Empty cells are also skipped. Both tests use a disarmed controller to inspect the visual and task-decision branches. An additional interface-injection test records the unsupported-class path. Original JSONL, console outputs, and evidence images are included. [21]

8.7 Red-cuboid and orange-bridge refinement

The detector includes a `red_orange_pair_classifier_v2.pt` stage for candidates labelled `red_cuboid` or `orange_bridge`. A result is accepted when its confidence is at least 0.80 and its lead over

the second class is at least 0.30. Otherwise, the main detector label is retained. This refinement changes the fine class while preserving the shared left-side sorting group. [22]

The classifier test produced 84/84 correct crops. In a video regression of 36 detections, 20 red-cuboid cases change from bridge labels to red-cuboid labels, and 16 bridge cases remain correct. Both evaluations used saved image data; the updated classifier has not been evaluated in a new physical acceptance run.

8.8 Operating settings

The main detector confidence is 0.25; NMS IoU is 0.45. Detection-box centers determine unique polygon ownership. A seven-observation history requires at least five matching sorting-group votes, with the current group also matching before a target is ready. The controller routes `cube` right and `other` left. [6, 11]

The classification gripper profile is `OPEN=100`, `cubeCLOSE=69`, `otherCLOSE=40`, and speed 20. The calibration files provide 17 measured reference poses and directional correction data. Before operation, the region layout and poses are checked against the current setup. [8, 9]

9 Main Problems and Solutions

Recognition and grid stability. A red/pink classifier reduced confusion between similar objects. Calibrated grid polygons and five-out-of-seven group voting stabilized cell selection. [4, 5, 6]

Position and gripper errors. We corrected taught poses and checked TCP and gripper feedback during transfer. Continued failure triggered recovery or shutdown. [7, 8, 9]

Task coordination. A control lock, request IDs and worker feedback kept transfers in order, with only one motion request active at a time. [6, 11]

Empty and unknown cells. Empty cells were skipped. A separate unknown-object test used occupancy comparison and an unsupported-class message to check that no movement request was issued. [12, 6]

Simulation extension. An unreachable bridge destination was repositioned, and constraint operations were moved to the Gazebo update thread to fix a release-stage crash. The task then passed 6/6. A MoveIt shutdown error after acceptance remains unresolved. [18, 19]

10 Conclusion

The system completed the required sequence from camera detection to grid selection, grasping and classified placement. Fixed pickup grids allowed us to use taught poses while leaving object selection and destination choice to the controller.

In simulation, four cubes and two bridges reached their assigned positions. On the physical arm, the instructor-approved test used five initial objects and one re-feed to complete six transfers. Original task logs record the six requests and completed results. Empty and unknown cells were checked in separate no-motion tests. [19, 20, 21]

The main implementation choices were five-of-seven group voting, one motion request at a time, and position and gripper checks during transfer. Red/pink and red/orange classifiers were added for the object pairs that caused recognition errors. The red/orange update has been evaluated on saved images and video. Further physical trials would be needed to measure its effect on repeated sorting runs.

A Physical Sorting Cycle

The physical sorting loop is outlined below. Motion routines handle retries and feedback checks.

Listing 1: Main stages of the complete physical task

```
Load the active models, regions, taught poses, and gripper profiles
Validate the configuration and start detector, controller, and worker
Acquire the robot control lock before armed operation
While the task is active:
    Update the stable states of all six source cells
    Log non-target states without requesting a movement
    If a transfer is running:
        Wait for feedback with the matching request ID
        On failure or timeout: stop the task and record the cause
    Else if a completed transfer needs source verification:
        Wait for the inter-object interval
        Accept EMPTY, or accept and requeue a stable ready cell
        Continue waiting for an unresolved source state
    Else if an eligible cell exists and the task is armed:
        Choose its group and destination automatically
        Send one transfer request to the worker
        Validate the selected route and open the gripper
        Move safe -> source approach -> source grasp
        Check the grasp pose, close, lift, and check the hold
        Move through safe to the selected destination
        Descend, release, verify clearance, and return to safe
        Report COMPLETED or FAILED with the motion-log path
    Else if work occurred and all cells stayed empty for 5 seconds:
        Complete the session
On exit: stop any active runner and close cameras and connections
```

A ready cell after transfer can mean either refill or an object left behind. The outline therefore preserves the current requeue rule without treating it as independent proof of physical success. The final placement must be checked using the video or another outcome measurement.

B Visual Processing Logic

The detector and controller logic is shown below. The robot worker executes the selected request. Here, UNKNOWN means an unsupported received label.

Listing 2: Visual processing and request selection in the supplied ROS pipeline

```
Load detector v4, red/pink v2, red/orange pair v2, and six polygons
Create a history of length 7 for each cell
For each camera frame:
    boxes = detect(frame, size=640, confidence=0.25, nms_iou=0.45)
    For each pink or red box:
        crop = crop_with_padding(frame, box, padding=0.08)
        label, score = classify(crop, size=224)
        If box is pink and label is red and score >= 0.55:
            change the class to red
    For each remaining red-cuboid or orange-bridge candidate:
        Run the red/orange pair classifier
        If confidence >= 0.80 and top-two margin >= 0.30:
            adopt the pair classifier label
    Publish classes, scores, box centers, and sizes
    Assign boxes to cells by unique center-in-polygon match
    For each cell:
        If no box: observation = EMPTY
        Else if an unsupported class exists: observation = UNKNOWN
        Else if known groups differ: observation = MULTIPLE
        Else: observation = shared group; choose highest-score box
        Update the seven-observation history
        If seven entries exist and at least five agree:
            stable = the majority group or state
        Else: stable = CANDIDATE
        A group is ready only if the current group also agrees
    If armed, idle, and the required waits are complete:
        Select the lowest-numbered ready cell
        Send one request: cube -> right; other -> left
```

C Submission File Index

The submission folder is `Group15_Experiment3_Compact_20260924`. The root `README_CN.md` describes the package contents and `SHA256SUMS.txt` records file checksums. Video encoding and original-file identities are listed in `VIDEO_COMPRESSION.md`.

Table 14: Submission directory guide

Material	Relative path	Contents
Group report	`01_Report/`	PDF, LaTeX source, and figures
Physical implementation	`02_Physical/`	ROS 2 detector, task controller, robot worker, hardware tools, Launch and deployment scripts
Models	`02_Physical/models/`	Main v4 detector, red/pink v2 classifier, red/orange pair v2 classifier
Classification settings	`02_Physical/config/class_mapping.yaml`, `gripper_profiles.yaml`	Class routing and gripper targets
Latest simulation	`03_Simulation/experiment3_sim_bridge_20260921/`	Six grids, four cubes and two bridges; code, videos and independent acceptance
Historical simulation	`03_Simulation/experiment3_sim_complete_20260914/`	Original normal and exception runs
Reviewed physical videos	`04_Evidence/Physical_Video/`	Acceptance MP4 and separate visual-recognition MP4 (compressed from AVI)
Original physical records	`04_Evidence/RawRuns/`	Six-cycle task, five-cycle v4 task, physical UNKNOWN and injection tests, console logs, images and videos
Task and video indices	`05_Records/`	Original request association index and video-cycle observations
Measured pose reference	`06_Calibration/measured_17poses_20260907.yaml`	Seventeen recorded robot poses for calibration reference
Direction compensation	`06_Calibration/joint_compensation.yaml`	Recorded directional correction data
Training and evaluation	`07_Training/`	Main detector training material and red/orange refinement datasets, scripts and evaluations

C.1 Reading the results

The final bridge simulation record is `03_Simulation/experiment3_sim_bridge_20260921/results/bridge_20260921_222016/acceptance.json`: six correct placements, including both bridges, with 37,831 joint samples. Its two videos and original event records are stored in the same run directory.

Physical request records preserve the original grid IDs, detector labels, destinations and worker feedback. The six-cycle and five-cycle runs retain separate identities. Video observations describe visible object transfers; offline classifier evaluations describe image-classification performance.

C.2 Configuration and use

The classification workflow uses main-detector confidence 0.25, seven-observation/five-vote sorting-group stability, unique center-in-polygon grid assignment, and cube/right versus other/left routing. Gripper values are OPEN 100, cube CLOSE 69, other CLOSE 40, and speed 20. Configuration examples and calibration tools accompany the source.

The measured-pose reference is retained in the calibration directory without replacing the robot's runtime configuration. Before physical operation, perform the documented region calibration, pose checks and low-speed validation for the current setup.

References

[1] Robot Integration Group Project I. *Experiment 3 Desktop Object Classification and Sorting: Simulation and Physical Verification Requirements*. Laboratory handout and report-format requirements, in Chinese.

[2] Group 15. *Experiment 3 Desktop Object Classification and Sorting*. Defense presentation, 10 slides. Visual module: slides 4–5; motion and results: slides 6–9. The chart on slide 9 records the v3 baseline.

[3] Zhang Feixiang. *Experiment 2 Fixed-Point Robotic Pick-and-Place*. Individual experiment report, 2026.

[4] Project visual implementation. `detector_node.py`, `experiment3_vision_preview.py`, and `class_mapping.yaml`. Training context: the local `cube_detection` project, including its README and `scripts/build_dataset.py`.

[5] Project grid calibration. `experiment3_region_calibrator.py`, `regions.example.yaml`, and the region-configuration README.

[6] Project task control. `experiment3_task_controller.py` and `experiment3_robot_worker.py`. ROS 2 implementation.

[7] Project teaching tools. `experiment3_pose_recorder.py`, `mecharm_real_recorder.py`, and `MANUAL_TEACHING_CN.md`.

[8] Project physical motion. `experiment3_transfer_test.py`, `experiment3_grid_dry_run.py`, `experiment3_grid_grasp_test.py`, `experiment3_all_grid_sequence.py`, and `mecharm_recorded_task.py`.

[9] Project gripper configuration. `gripper_profiles.yaml`, `experiment3_gripper_profiles.py`, and `experiment3_gripper_profile_tester.py`.

[10] Project optional joint correction. `experiment3_directional_compensation.py`.

[11] Project deployment and startup. `experiment3_real.launch.py`, the complete-system README, Windows deployment and synchronization scripts, and `start_experiment3_ros2_auto.cmd`.

[12] Project exception tests. `experiment3_unknown_exception_test.py`, `experiment3_physical_unknown_test.py`, and their corresponding `run_experiment3_*_test.sh` launch scripts.

[13] Project simulation and keyframe base. `experiment.launch.py`, `teaching.launch.py`, `task_executor.py`, `keyframe_player.py`, `task_points.yaml`, Gazebo world/model files, and the simulation README.

[14] Project saved classifier evaluation. `cube_detection/models/red_pink_classifier_v2_metrics.json`. Two-class test split with 26 pink and 42 red crops; reported Top-1 accuracy 94.12%.

[15] Group 15. Baseline simulation source, `experiment3_sim_complete_20260914`: Launch, configuration, detector, controller, grasp bridge, motion and acceptance monitor.

[16] Group 15. Baseline records in `experiment3_sim_complete_20260914/results/normal_front_arc_04/` and `exceptions_front_arc_03/`: acceptance, logs and videos for normal sorting and designated exceptions.

[17] Group 15. Combined Experiment 2/3 core-code archive, dated 2026-09-13. Physical code in `02_Physical/`, including ROS packages, detector models, class mapping, gripper profiles and deployment tools.

[18] Group 15. Six-grid bridge extension, `experiment3_sim_bridge_20260921`: configuration, world generator, grasp plugin, detector, motion and retained development runs.

[19] Group 15. Final bridge run, `experiment3_sim_bridge_20260921/results/bridge_20260921_222016/`: independent acceptance, configuration, geometry, per-job evidence and two camera videos.

[20] Group 15. Physical video records in `04_Evidence/Physical_Video/`: visual-recognition demonstration `acceptance_20260907_192601.mp4` (compressed from the original AVI), and acceptance video `2eb7001d38fd861630d52918161ba8cf.mp4`. Original identities and encoding details: `VIDEO_COMPRESSION.md`.

[21] Group 15. Physical task JSONL, console logs, and UNKNOWN-test images. Six-cycle task: 20260908_032515; five-cycle v4 task: 20260907_192601. Included in `04_Evidence/RawRuns/`; request association in `05_Records/task_index.json`.

[22] Group 15. Red-cuboid/orange-bridge pair classifier v2, detector integration, and saved offline evaluation reports. Code and weights in `02_Physical/`; training and regression evidence in `07_Training/`. Crop test: 84/84; video regression: 36/36 evaluated detections.